



Threat Modeling for EquiTask

Scope: Task Data & Proof Data

1 What Are We Protecting? (Assets)

Primary Assets

1. User Account Data

- Email
- Password hash
- Role (Admin/User/Manager)

2. Task Data

- Task title
- Description
- SMART fields
- Deadlines
- Completion status

3. Proof Data (High Risk)

- Uploaded images
- Audio recordings
- Text explanations
- Metadata (timestamps, device info)

4. Behavioral Analytics

- Completion patterns
- Delay history
- Performance metrics

Proof data is the **highest sensitivity asset** because:

- It may contain personal photos
 - It may reveal work environment
 - It may contain private company information
-

2 Threat Modeling Framework (Simplified STRIDE)

We'll analyze using simplified categories:

- Spoofing
 - Tampering
 - Repudiation
 - Information Disclosure
 - Denial of Service
 - Privilege Escalation
-

3 Key Threats & Mitigations

● 1. Spoofing (Fake Identity Access)

Threat:

An attacker logs in as another user and accesses their tasks and proof.

Risk Level: High

Mitigation:

- JWT-based authentication
- Secure password hashing (bcrypt/argon2)
- Token expiration & refresh strategy
- Optional 2FA for admins
- Rate limiting on login endpoint

Backend Requirement:

- Never trust client-side identity
- Validate JWT on every protected route

● 2. Unauthorized Access to Proof Files

Threat:

Someone accesses uploaded images via direct URL.

Example:

<https://storage.equitask.com/proofs/image123.jpg>

If storage is public, data leak occurs.

Risk Level: Critical

Mitigation:

- Store proofs in private cloud storage
- Use signed URLs with expiration
- Enforce role-based access checks
- Validate ownership before returning proof

Backend Rule:

User must only access proofs tied to their user ID or team scope.

3. Data Tampering

Threat:

User modifies task completion status through API manipulation.

Example:

Changing request payload manually.

Risk Level: Medium

Mitigation:

- Validate all input server-side
- Enforce schema validation (Joi/Zod)
- Restrict status changes to authorized endpoints
- Audit log changes

Never rely on frontend validation alone.

4. Malicious File Upload (Very Important)

Threat:

User uploads:

- Malware disguised as image
- Script file renamed as .jpg
- Extremely large file to crash system

Risk Level: High

Mitigation:

- Validate MIME type server-side
- Restrict file size (e.g., max 10MB)
- Virus scan uploads (if possible)
- Strip metadata from images
- Rename files with UUID (never trust original filename)

Backend MUST:

- Reject unknown file types
- Reject executable extensions

5. Information Disclosure via Analytics

Threat:

Manager views data beyond their scope.
Cross-team data leakage.

Risk Level: Medium

Mitigation:

- Role-based access control (RBAC)
- Enforce team scoping at query level

- No global queries without filtering
-

6. Data Leakage via Logs

Threat:

Sensitive data appears in server logs.

Example:

- Proof URLs
- Tokens
- Password hashes

Risk Level: High

Mitigation:

- Never log tokens
 - Never log full request bodies for proof upload
 - Mask sensitive fields in logs
-

7. Denial of Service (DoS)

Threat:

Massive proof upload attempts to overload server.

Mitigation:

- Rate limiting
- File size restriction
- Upload throttling

- CDN for file storage
-

8. Privilege Escalation

Threat:

User changes role field in API request to “admin.”

Mitigation:

- Role must be assigned server-side only
 - Never allow role updates from client request
 - Validate role from JWT, not request body
-

Data Flow Overview (For Developers)

Mobile App

- HTTPS
- Backend API
- Database (Tasks, Users)
- Secure Object Storage (Proof Files)

All communication:

- HTTPS only
 - TLS enforced
 - CORS configured properly
-

5 Security Controls Checklist (Backend Team)

Authentication

- JWT with expiration
- Password hashing
- Rate limiting login

Authorization

- RBAC
- Validate ownership on every task/proof fetch

Storage

- Private cloud bucket
- Signed URL access
- Encrypted at rest

Upload Protection

- File type validation
- File size limit
- Virus scanning (if possible)

API Security

- Input validation

- Error message sanitization
 - No stack trace exposure
-

6 Highest Risk Areas (Priority Focus)

If time is limited, protect these first:

1. Proof file storage access control
2. File upload validation
3. Authentication & token validation
4. Role-based access enforcement

These are your “must not fail” areas.

7 Simple Summary for Non-Security Members

EquiTask handles sensitive proof data.

If compromised, users could:

- Have private images exposed
- Lose trust in the platform
- Experience identity misuse

So we must:

- Secure authentication

- Restrict access to task/proof data
- Protect file uploads
- Prevent unauthorized access
- Encrypt and validate everything

Security is not optional — it is core to inclusion and trust.